

Multilevel Hypergraph Partitioning

实验结果与论文结论

依据 Karypis et al., IEEE TVLSI, 1999

学习日期：2026 年 9 月 15 日

今日目标

今天精读论文第 IV 节与第 V 节，学会“像审稿人一样读表”。完成后，你应当能够：

1. 说明 45–55 平衡条件与 best-of- N 报告口径的含义；
2. 解释 Table II 中每一个 hMETIS 列代表什么实验配置；
3. 区分“总 cut”“逐电路平均改善百分比”“Best 列”三种比较方式；
4. 用论文给出的数字说明 FM、FM-EE 与多得化 (V-cycle) 各带来多少收益；
5. 解释为什么超图多层方法明显优于基于图的 GMetis；
6. 复述论文对多层范式成功原因的三点总结。

建议学习时长：100–120 分钟。本日对应论文第 IV、V 节。

目录

1	从第四天接上：为什么必须会读实验表	3
2	实验设置：先读懂约束与口径	3
2.1	硬件与测试集	3
2.2	平衡条件：强制 45–55	3
2.3	四个 hMETIS 列分别是什么	4
3	Table II 的结构：三类比较对象	4
4	关键数字一：hMETIS-EE20 相对早期算法	5
5	关键数字二：FM 与多得化各值多少	6
6	关键数字三：运行时间与“跨机器不可直接比较”	6
7	关键数字四：规模越大，优势越明显	6
8	为什么超图方法明显强于基于图的 GMetis	7
9	可扩展性：为什么“大图更划算”	7

10 结论：多层范式为什么有效	8
11 如何把这套评估方法用到自己的实验	8
13 第六天预告	9

1 从第四天接上：为什么必须会读实验表

前四天回答了“算法怎么工作”：

粗化 → 最粗层初始划分 → 反粗化细化 → 多阶段细化.

今天要回答“这些设计到底值多少”。论文第 IV 节用一张对比表加一张效率图给出证据。读表时要时刻分清三件事：

读表三问

1. **列是什么配置**：用 FM 还是 FM-EE？是否使用多得化 (V-cycle)？跑了几次？
2. **比较口径是什么**：总 cut 之和、逐电路平均改善百分比，还是与“已知最好解”比较？
3. **代价是什么**：同一张表下方还有运行时间子表，质量收益必须与时间收益一起看。

2 实验设置：先读懂约束与口径

2.1 硬件与测试集

- 平台：SGI Challenge, MIPS R10000 处理器, 200 MHz；所有时间以秒计；
- 测试集：广泛使用的 ACM/SIGDA 电路划分基准集合，共 23 个超图；
- 规模跨度大，最大者含 10 万以上顶点（结论中明确提到 100 000 顶点以上的超图也能在几分钟内得到高质量二分）。

2.2 平衡条件：强制 45–55

论文所有报告结果都强制

两侧相对规模必须在45% 到55% 之间.

这条约束非常重要：

- 它保证比较是“同平衡度下比 cut”，否则把顶点大量堆到一侧总能减小 cut，比较就没有意义；
- 它同时限制了 FM/HER 的可行移动，第 3 天讲过的“最高 gain 也可能不可移动”正是被它拦下；
- 45–55 允许两侧偏差约 10%，比严格的 50–50 更宽松，给局部搜索留出腾挪空间。

2.3 四个 hMETIS 列分别是什么

列名	运行次数	配置说明
hMETIS-EE20	20	细化使用 FM-EE；20 次中 10 次用 HEC、10 次用 MHEC 粗化；最粗层用随机初始划分
hMETIS-FM20	20	细化使用标准 FM；粗化配置同上（即仅把 FM-EE 换成 FM）
hMETIS-EE10vV	10	使用多得化（V-cycle）细化方案；粗化用 MHEC，细化用 EE 版 FM
hMETIS-FM20vV	20	使用 V-cycle 方案；10 次 HEC、10 次 MHEC；细化用 FM

为什么报 best-of- N 而不是平均值

多层算法三个阶段都含随机性，单次运行的结果波动较大。论文的目标是回答“给定合理的计算预算，该方法能拿到多好的解”，因此报告多次运行中的最好结果；结论也明确说：即便单次运行已能得到相当好的二分，用小量次数（例如 20 次）就能超过以往所有已知算法。阅读时要注意：这不等同于“平均质量”，而是“搜索能力上限”。

3 Table II 的结构：三类比较对象

比较对象	含义与读法
早期算法各列	PROP、两个早期细化类方案、Optimized KLFM、GMetis、PARABOLI、GFM 等，各自报告自己的最好结果
Best 列	每个电路上所有早期算法中的最小 cut。它相当于“假如把所有早期算法都跑一遍再逐电路取最优”，是很强的对手
hMETIS 四列	本论文四种配置，见上一节表格

论文的比较方式是：先算各算法在所有电路上的 **总 cut**，再算**逐电路平均改善百分比**。后者更公平，因为大电路 cut 绝对值大，若只看总和，结果会被少数大电路主导。

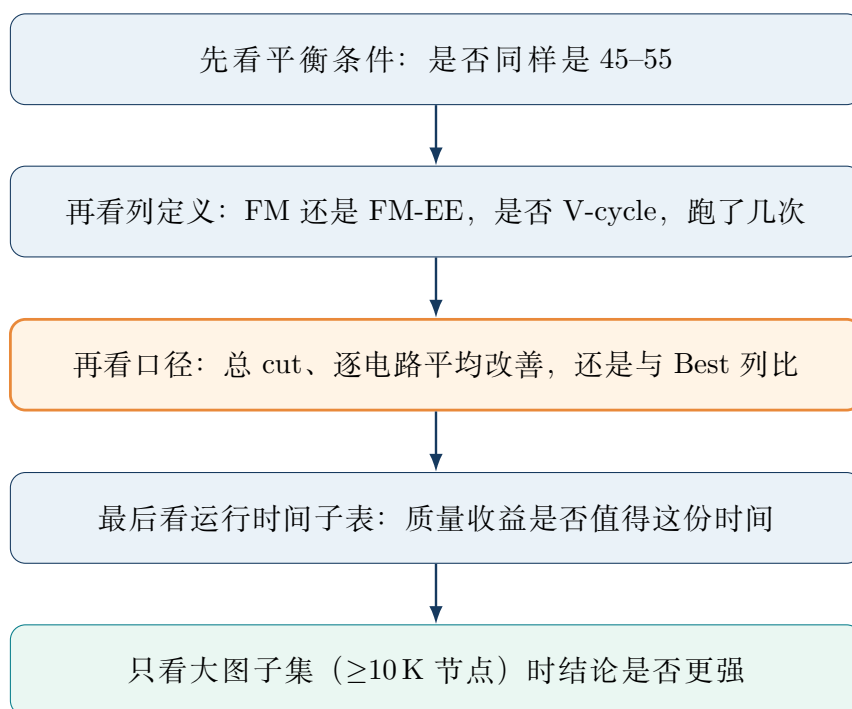


图 1: 审稿人式读表顺序：配置 → 口径 → 代价 → 规模分层

4 关键数字一：hMETIS-EE20 相对早期算法

论文给出的逐电路平均改善如下（相对值越大越好）：

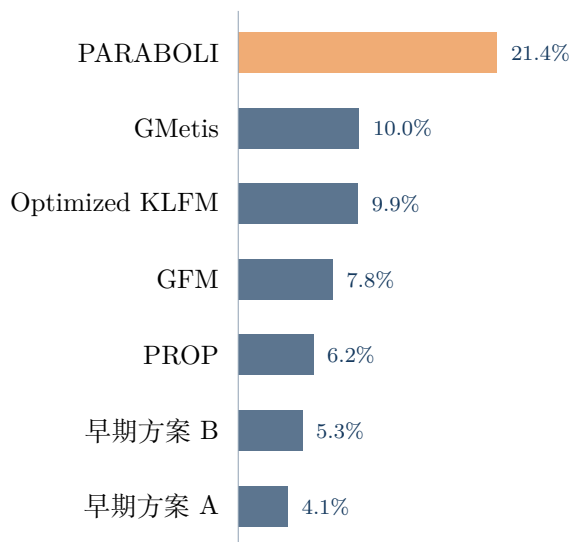


图 2: hMETIS-EE20 在全部 23 个电路上相对各早期算法的平均 cut 改善

要点：

- 对所有列出的早期算法，hMETIS-EE20 都更好；
- 即使与 **Best 列**（各电路上早期算法的最好者逐电路取最优）比较，仍然好 0.3%，说明它不只是“打败了某一个算法”，而是整体优于这一批方法；

- 改善幅度最大的对象是 PARABOLI (21.4%)，最小的是两个早期细化类方案 (4.1% 与 5.3%)，因为后两者也使用了较强的细化技术。

5 关键数字二：FM 与多得化各值多少

比较	论文结论
FM 相对 FM-EE	hMETIS-FM20 约比 hMETIS-EE20 好 1.1%，比所有早期方案合并比较还好 1.4%；并在 23 个电路中的 8 个改进了当时的已知最好二分
多得化相对单次流程	hMETIS-FM20vV 比 V-cycle 的 EE 版本好约 2.0%，比 hMETIS-FM20 好 0.9%，比 Best 列好 2.3%
总体最佳配置	hMETIS-FM20vV 是论文给出的整体最优方案
可以读出两个设计判断：	

- 完整 FM 比早停 FM 更值：省时间的 FM-EE 在质量上大约让出 1%；
- 多得化 (第四天的 V-cycle) 是有效的质量工具：在相同细化算法下再拿回收约 2%，这正说明“换一种随机粗化从而改变可用的成组移动”确实能跳出局部最优。

6 关键数字三：运行时间与“跨机器不可直接比较”

论文的时间来自不同机器，作者明确指出这使直接比较困难，并给出换算参考：自家代码在 Sparc5 上所需时间约为 R10000 的 4 倍。按这一缩放因子解读：

- hMETIS-EE20 比 PROP 快约 4 倍，比两个早期方案快约 9 倍，比 PARABOLI、GFM、Optimized KLFM 快得多；
- 与 GMetis 相比，hMETIS-EE20 耗时大致相当，hMETIS-FM20 约慢两倍；
- 但 GMetis 的时间是“对每张图跑 100 次 METIS”的结果，而每次 METIS 运行本身比 hMETIS 快得多——部分因为 METIS 是高度优化的图代码，部分因为超图的粗化与细化本身比图更复杂；
- 关键在于：即便这样比较，两个 hMETIS 配置切开的超边都明显少于 GMetis。

另一个反直觉的结论是：使用多得化后总时间反而下降。

hMETIS-EE10vV : 409 s vs hMETIS-FM20vV : 1513 s

即四个配置中最省时的不是最简单的单次流程，而是 EE 细化加多得化的组合。原因与第 4 天的分析一致：W/V 类方案把宝贵的细化预算从“多个候选 × 大图”转移到了更便宜、收益更高的层次。

7 关键数字四：规模越大，优势越明显

只看 10 K 节点以上的大电路时，hMETIS-FM20vV 相对各早期算法的改善变成：

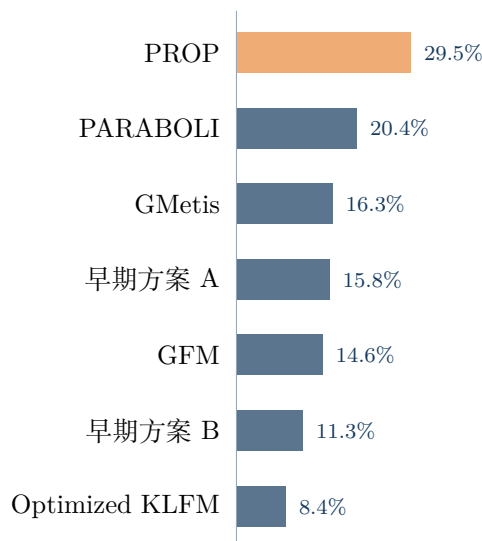


图 3: hMETIS-FM20vV 在大电路子集 ($\geq 10K$ 节点) 上的平均改善

同一图表在全部电路与仅大电路上的数值差异, 说明分层结论很重要: **多层超图方法的相对优势随问题规模增大而扩大**。最直观的极端例子是最大的基准 `golem3`: GMetis 跑 100 次所得的最好结果, 仍比 hMETIS-FM20vV 的 10 次运行差约 50%。

8 为什么超图方法明显强于基于图的 GMetis

论文给出两条原因, 都直接指向前面几天的技术选择:

1. **粗化方式**: 超边粗化让最粗层“暴露在外的超边权重”下降得更多, 因而得到远比基于边的粗化更好的初始划分;
2. **优化目标**: 细化阶段直接最小化超边 cut, 而不是在不精确的图近似上最小化边 cut。

换言之: GMetis 先把超图转成图, 再套用图多层方法; hMETIS 全程在超图上工作, 既避免了近似误差, 也让聚合单位与真实代价一致。

这张表与第一天笔记的呼应

第一天我们说“把超图转成 clique 图会放大规模并扭曲代价”。今天 GMetis 的对比结果正是这一判断的实验版本; 而 hMETIS 通过隐式 clique 权重 (第 2 天 EC 的 $1/(|e| - 1)$) 与整边收缩 (HEC/MHEC) 在超图上直接工作。

9 可扩展性: 为什么“大图更划算”

论文对扩展性给出三点归纳:

1. 粗化阶段的时间与超图规模成正比, 因此总运行时间随超图规模近似线性增长;
2. 由于多层范式的性质, **获得高质量二分所需的运行次数不随规模增长**——这一点与其他启发式 (往往需要更多随机重启) 形成对比;
3. 10 万以上顶点的超图可在当时的几十分钟工作站上几分钟内得到高质量二分。

10 结论：多层范式为什么有效

论文把成功原因归为三层分工：

1. **粗化提供全局视角**：好的粗化方案构造出的粗超图是原问题的良好近似，使最粗层能在“全局”而非局部尺度上做决定；
2. **初始划分利用全局信息**：在最粗层求得的二分相当于在模块级一次决定大方向；
3. **逐层细化提供局部视角**：反粗化时每次移动的是“越来越小的顶点子集”，因而能在保持全局结构的前提下持续改善边界。

全局视角定大方向	+	逐层局部细化修边界	=	多层范式
----------	---	-----------	---	------

作者同时给出未来方向：寻找适用于更广泛超图类的粗化方案；把更强的 FM 变体引入反粗化，但需权衡其开销是否值得——因为“多跑几次多层算法”本身也是一种有效的质量投资。

11 如何把这套评估方法用到自己的实验

按论文口径复现一次公平对比，建议至少交代以下信息：

1. **平衡约束**：报告 45-55 还是 50-50，以及是否所有算法都遵守；
2. **运行次数与报告方式**：best-of- N 、平均、还是中位数； N 是多少；
3. **改善的定义**：逐电路平均，还是总 cut 之比；是否给出逐电路明细；
4. **对手口径**：是否包含“各对手逐电路取最优”的 Best 列；
5. **时间口径**：机器是否可比，是否给出缩放换算；
6. **规模分层**：是否单独报告大图子集结果，避免被小图平均掉。

常见误读提醒

- 把 best-of-20 与对手单次结果比较，会高估自己的优势；
- 只报总 cut，会让少数大电路主导结论；
- 忽略平衡条件差异，等于在比“谁切得少”而不是“谁在同等约束下切得少”；
- 跨机器直接比秒数，会得出错误的效率结论。

13 第六天预告

下一天作为本论文的收尾：把五天内容压缩成一张能默写的全景图，讨论 hMETIS 之后的演进方向（如更强的 FM 变体、K-way 划分、并行多层划分），并设计一个可执行的小规模复现实验方案，用自己写的简化实现跑通“粗化-初始划分-反粗化细化”全流程。

来源与精度说明

本文依据论文第 IV、V 节整理。论文该处有两个算法名使用特殊字体排版，纯文本提取无法辨识，故图中记为“早期方案 A/B”（分别对应论文参考文献中的两个早期细化类方案），其改善数值按论文给出的顺序为 4.1%、5.3%（全部电路）与 15.8%、11.3%（大电路子集）。运行时间跨机器不可直接比较，本笔记只复述论文给出的 Sparc5 与 R10000 之间约 4 倍的缩放说明。